

Library Management System

Java Swing Desktop GUI with Flat-File Persistence

Student Name:	Abhinav Singh
Registration Number:	25BAI10303
Course:	Programming in JAVA
Submission Date:	18/09/2026
Primary Source File:	LibraryManagement.java
Platform / Portal:	VITYarthi

What is Used in the Code:

Category	Java Classes & Packages Used	Role & Implementation in Code
GUI Framework	javax.swing.* (JFrame, JTabbedPane, JPanel, JTable, JScrollPane, JTextField, JButton, JLabel, JOptionPane)	Constructs main window, tabs, forms, data tables, scrollbars, and alert dialogs.
Table Model	javax.swing.table.DefaultTableModel java.util.Vector	Dynamic in-memory representation of table rows; handles row appending, clearing, and lookups.
Layout Managers	java.awt.BorderLayout java.awt.GridLayout java.awt.FlowLayout	Responsive visual structure: form grid layouts, flow button strips, and border panel docks.
File I/O Streams	java.io.File java.io.FileReader, BufferedReader java.io.FileWriter, PrintWriter	Handles file checks, line-by-line CSV streaming, append writes, and temp-file replacement.
Look & Feel	javax.swing.UIManager	Sets host OS native UI theme via <code>getSystemLookAndFeelClassName()</code> .
Concurrency	javax.swing.SwingUtilities	Ensures GUI thread safety by invoking startup on Event Dispatch Thread (EDT).

1. Introduction

The **Library Management System (LMS)** is an autonomous desktop software solution developed for the **VITyarthi project** submission under the **Programming in JAVA** course. Authored by **Abhinav Singh (Reg. No: 25BAI10303)**, the project delivers an accessible, efficient cataloging and membership tool tailored for departmental book collections, collegiate reading rooms, and personal libraries. Developed with pure Java Standard Edition components (Swing GUI and Java File I/O), the system operates entirely driverless, requiring no external database server installations.

2. Problem Statement

Small-to-medium library environments frequently encounter practical dilemmas with conventional recordkeeping:

1. Physical Ledger Vulnerability: Manual paper registers are susceptible to physical wear, lack searchability, and cannot ensure synchronized record consistency.

2. Enterprise ILS Overhead: Industrial Integrated Library Systems (such as Koha or DSpace) require complex SQL database setup (MySQL/PostgreSQL), network connectivity, server hardware, and routine administrative overhead.

Project Objective: Implement an accessible, zero-dependency, driverless desktop tool in Java Swing that guarantees rapid CRUD operations, prevents record corruption via atomic temp-file swapping, and provides a platform-native user interface.

3. Functional Requirements

Req ID	Module	Requirement Specification	Priority
FR-01	Book Catalog	Capture Book ID, Title, and Author; enforce non-empty Book ID and Title.	Critical
FR-02	Book Catalog	Support in-place metadata updates (Title, Author) via modal prompts upon row selection.	High
FR-03	Book Catalog	Remove selected book records from in-memory table and books.txt.	Critical
FR-04	Members	Register patron Member ID and Name; reject records with empty parameters.	Critical
FR-05	Members	Delete selected member records from memory and members.txt.	High
FR-06	Persistence	Auto-initialize books.txt and members.txt on initial program launch.	Critical
FR-07	Persistence	Stream CSV records directly into DefaultTableModel upon boot and table events.	Critical

4. Non-Functional Requirements

Dimension	Quality Attribute	Engineering Criterion
Portability	JVM Independence	Runs across any standard Java Runtime (JRE 8+) on Windows, Linux, and macOS.
Fault Tolerance	Data Preservation	Atomic temp-file swapping (temp.txt -> target) prevents truncation during power cuts.
Usability	Ergonomic GUI	Native OS Look-and-Feel integration via UIManager; dual-tab single-window workflow layout.
Efficiency	Low Resource Footprint	Sub-50MB RAM footprint; sub-50ms latency on file reads up to 5,000 records.

6.3 Sequence Diagram: Book Insertion & Persistence

```

User/Operator LibrarySystemGUI DefaultTableModel File System (books.txt) | | | | -- Click Add -----> | | | (ID,
Title, Auth) | -- Validate Inputs | | | | -- appendToFile(record) -----> | (Append CSV) | | <--
Acknowledge Write -----> | | | | -- loadData() -----> | (Read All)
| | <-- Return Stream -----> | | | | -- setRowCount(0) -----> | | | | -- addRow(Vector) -----> | |
| -- Clear TextFields | | <-- Visual Update -|

```

6.4 Class & Component Architecture

```

+-----+ | LibrarySystemGUI |
+-----+ | - BOOKS_FILE: String =
"books.txt" | | - MEMBERS_FILE: String = "members.txt" | | - bookTableModel: DefaultTableModel | | -
memberTableModel: DefaultTableModel | | - bookTable: JTable | | - memberTable: JTable |
+-----+ | + LibrarySystemGUI() |
| - initFiles(): void | | - createBookPanel(): JPanel | | - createMemberPanel(): JPanel | | - loadData(filename:
String, model: DefaultTableModel, columnCount: int): void | | - appendToFile(filename: String, data: String): void |
| - deleteRecord(filename: String, idToDelete: String): void | | - updateRecord(filename: String, idToUpdate: String,
newRecord: String): void | | + main(args: String[]): void |
+-----+

```

6.5 Entity-Relationship Schema (Flat-File Representation)

Entity (File)	Attribute	Type	Key Type	Integrity Rule
books.txt	Book ID	String	Primary Key	Mandatory unique token
	Title	String	None	Mandatory description
	Author	String	None	Optional author attribution
members.txt	Member ID	String	Primary Key	Mandatory unique token
	Name	String	None	Mandatory patron name

7. Design Decisions & Rationale

Architectural Decision	Design Alternative	Engineering Rationale
Flat-File Delimited Storage	SQLite / H2 Embedded DB	Eliminates external JDBC binaries; ensures compilation and execution on vanilla JRE.
Atomic Temp-File Buffering	RandomAccessFile in-place writes	Guarantees atomic file writes; avoids file truncation in the event of JVM interruption.
Tabbed Panel Layout	Multi-Frame (MDI) Desktop	Minimizes window management overhead and keeps workflows in a single viewport.
Native OS Look-and-Feel	Default Metal theme	Integrates directly with host desktop rendering (Windows, macOS, Linux).

8. Implementation Details

The codebase is consolidated within LibrarySystemGUI.java, structured into modular procedures:

- 1. System Bootstrapping:** initFiles() verifies or creates books.txt and members.txt using File.createNewFile(). The UI Look-and-Feel is initialized via UIManager.setLookAndFeel.
- 2. Form Assembly:** createBookPanel() and createMemberPanel() construct responsive UI layouts using BorderLayout, nested GridLayout, and FlowLayout containers.
- 3. Dynamic Data Synchronization:** loadData() executes an O(N) stream read using BufferedReader, clearing existing rows via model.setRowCount(0) and appending freshly tokenized records.
- 4. Fault-Tolerant Mutation:** updateRecord() and deleteRecord() read source lines, filter against the target key, write output to temp.txt, and complete the modification via atomic file renames.

9. Screenshots / Results (UI Wireframe Representation)

```
+-----+ | Library Management System - [ ] X |
+-----+ | [Manage Books] | Manage Members | |
| | | Book ID: [ B-101 ] | | Title: [ Design Patterns ] | | Author: [ Gang of Four ] | | | [ Add Book ] [ Update
Selected ] [ Delete Selected ] | |-----| |
Book ID | Title | Author | |-----+-----+-----+-----| | B-101 |
Design Patterns | Gang of Four | | B-102 | Clean Code | Robert C. Martin | | B-103 | The Pragmatic Programmer | Andrew
Hunt | +-----+
```

10. Testing Approach & Verification Matrix

Test ID	Description	Input Data	Expected Outcome	Status
TC-01	File Initialization	Launch with absent files	Auto-generates books.txt and members.txt.	PASS
TC-02	Book Input Validation	Blank ID & Title	Displays warning dialog; blocks persistence.	PASS
TC-03	Book Insertion	B-101, 'Design Patterns', 'GoF'	Appends record to disk; refreshes JTable.	PASS
TC-04	Book Record Update	Select B-101; update Title	Replaces record using temp.txt atomic swap.	PASS
TC-05	Book Record Deletion	Select B-101; click Delete	Purges record from table and books.txt.	PASS
TC-06	Member Insertion	M-001, 'Abhinav Singh'	Appends record to members.txt; renders in table.	PASS
TC-07	Cold Restart Reload	Restart JVM process	Populates tables directly from saved disk state.	PASS

11. Challenges Faced

- 1. OS-Level File Locking on Windows:** On Windows environments, open file descriptors block atomic deletions. Enforcing strict try-with-resources blocks was necessary to guarantee stream closure before calling File.delete() and File.renameTo().
- 2. CSV Token Inconsistencies:** Text entries containing comma characters can disrupt column parsing. Defensive vector sizing within loadData() was implemented to guard against ArrayIndexOutOfBoundsException.
- 3. Swing Concurrency & Thread Safety:** Modifying Swing UI components outside the Event Dispatch Thread (EDT) can introduce race conditions. The system ensures safe startup via SwingUtilities.invokeLater().

12. Learnings & Key Takeaways

- **Separation of Concerns:** Isolating data retrieval and storage operations from the UI layout simplifies code maintenance and testing.
- **Atomic Operations in File Systems:** Temporary buffer files provide a straightforward, effective strategy for preventing partial or corrupt writes in desktop environments.
- **Defensive User-Input Validation:** Validating inputs before write operations preserves data integrity and reduces downstream runtime exceptions.

13. Future Enhancements

Roadmap Phase	Feature Enhancement	Architectural Evolution
Phase 2	Issue / Return Borrowing Ledger	Implement transactions.txt to manage loans, due dates, and return states.
Phase 3	Real-Time Filter & Search	Bind TableRowSorter to DefaultTableModel for keyword queries.
Phase 4	Embedded SQL Integration (SQLite)	Migrate flat-file logic to SQLite via JDBC transactions.

14. References

1. Oracle Corporation. *Creating a GUI With JFC/Swing: The Java Tutorials*. Oracle Documentation, 2024.
2. Bloch, Joshua. *Effective Java (3rd Edition)*. Addison-Wesley Professional, 2018.
3. Gamma, Erich, et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
4. Eckel, Bruce. *Thinking in Java (4th Edition)*. Prentice Hall, 2006.
5. Deitel, Paul, and Harvey Deitel. *Java: How to Program (Early Objects, 11th Edition)*. Pearson, 2017.